

MARITIME COMPLIANCE AI PLATFORM

ARCHITECTURAL DESIGN DOCUMENT

DOCUMENT CONTROL

- **System Version:** 1.0.0
 - **Architecture Paradigm:** Decoupled MCP Gateway with Persistent Vector Storage
 - **Core Inference Engine:** LangGraph State Machine & Llama 3.1 (Groq Network Execution)
-

1. EXECUTIVE SUMMARY & ARCHITECTURAL PHILOSOPHY

The Maritime Compliance Platform is an enterprise-grade software solution built to resolve complex, cross-domain safety and regulatory alignment queries across **MARPOL Regulations, SIRE 2.0 Inspection Checklists, and ISM Code** frameworks.

Historically, LLM-driven compliance tools suffer from high vocabulary fragility and hallucinations due to an over-reliance on brittle, raw keyword matching layers. This platform eliminates these limitations by establishing a strict **Decoupled Architecture Paradigm**.

By separating the network/transport layer from the underlying data math computation systems, the platform delivers three core architectural guarantees:

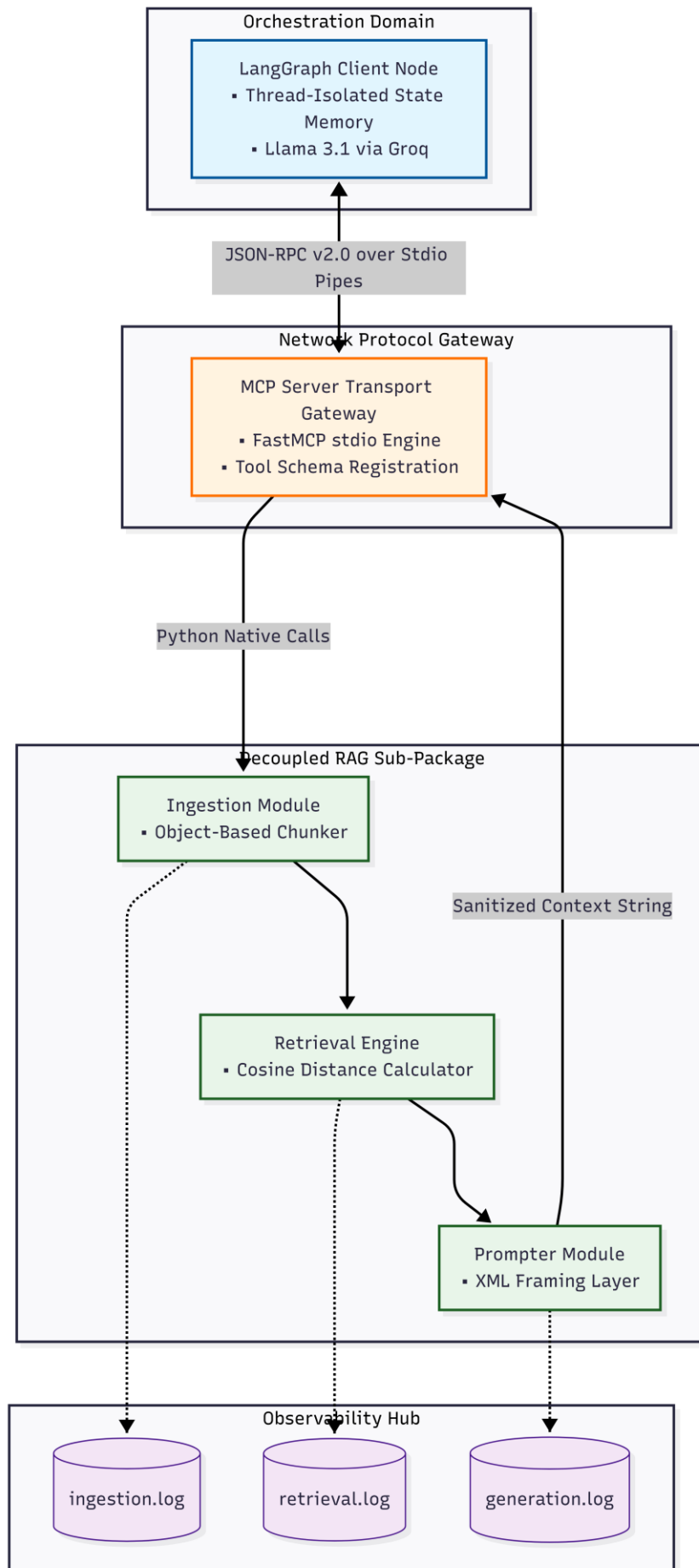
- **Strict Separation of Concerns:** The Model Context Protocol (MCP) server environment operates purely as a stateless, isolated data router. It contains no data structures, file path dependencies, or embedding calculations.
- **Deterministic Execution Safeguards:** The orchestration graph features automated run-loop safety switches. It uses thread-isolated conversational state boundaries to completely eliminate multi-tool calling cascades or context leakage between evaluation intervals.
- **Cross-Platform Environment Portability:** The underlying storage layout utilizes an automated self-healing caching database system. It delivers lightning-fast local read performance without creating hard operating system path-locks.

2. HIGH-LEVEL DESIGN (HLD) & CORE TOPOLOGY

The overall topology organizes execution environments into distinct, vertically bounded layers. Machine-to-machine interactions are managed securely over standard input/output (stdio) pipe streams using structured **JSON-RPC v2.0** communication protocols.

2.1 Component Architecture Map

- **Orchestration Domain (Client Node):** Manages user multi-turn dialogs and intercepts tool outputs using an isolated LangGraph state execution machine. It routes system prompts directly through the Groq API network to handle Llama 3.1 inference sweeps.
- **Network Protocol Gateway (MCP Server):** Exposes programmatic tool schemas to the orchestration layer. It remains completely stateless, executing tool triggers by invoking the decoupled data core as a standard background Python package dependency.
- **Decoupled RAG Component (Data Core):** An isolated software library packaged within the repository. It handles the object-based normalizers, local vector coordinate maps, and mathematical distance threshold filtering.
- **Observability Hub (Telemetry Layer):** A dedicated logging directory that pipes telemetry data streams directly to distinct, unlinked text files. This setup exposes the exact mathematical operations occurring behind your user queries, preventing silent RAG failure modes.



2.2 Data Flow Lifecycle

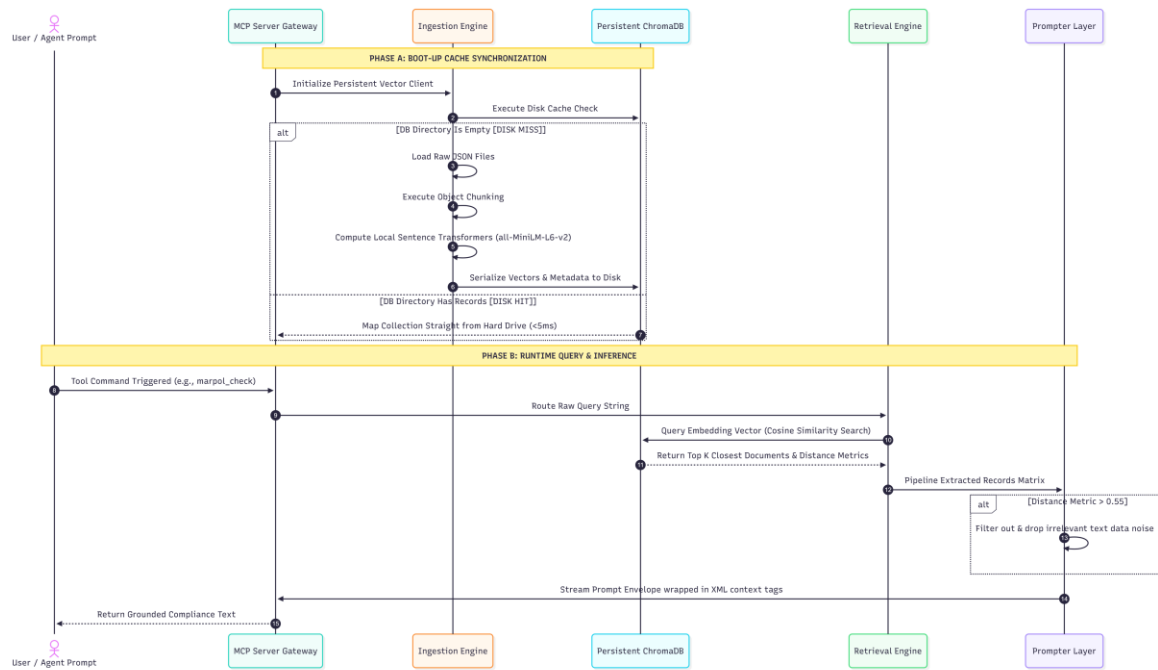
The system's operational lifecycle executes through two standalone operational phases to guarantee data validity and lower execution latency.

Phase A: Boot-Up Cache Synchronization

1. The MCP Server initializes the `PersistentVectorStore`.
2. The Ingestion Engine performs an automated disk-check on the local tracking directory (`rag/embeddings_db/`).
3. **If a DISK MISS occurs:** The system reads raw JSON records from `/data`, applies object-based contextual chunking, computes 384-dimensional array vectors via the local `all-MiniLM-L6-v2` transformer, and serializes the assets to disk.
4. **If a DISK HIT occurs:** The engine bypasses calculations entirely, mounting the active collections directly from disk in under 5 milliseconds.

Phase B: Runtime Tool Query & Inference

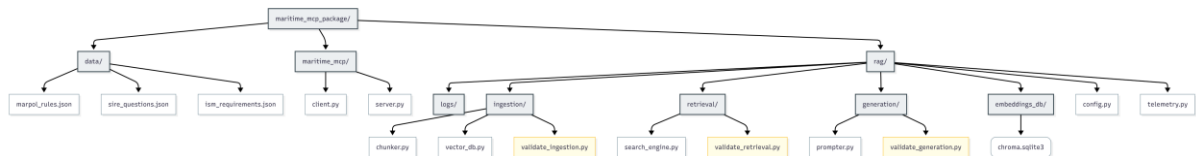
1. A user prompt is submitted to the LangGraph application, which forwards the payload to Llama 3.1 via Groq.
2. The model detects regulatory intent and generates an explicit MCP tool call request (`marpol_check`).
3. The MCP Server receives the transport schema and passes the query string to the decoupled `rag` package.
4. The `search_engine` converts the input to an embedding vector, executes a local cosine similarity search against the persistent disk space, and pipelines the top matches to the prompter.
5. The `prompter` filters out chunks with an irrelevant semantic distance (greater than 0.55), wraps the surviving text blocks inside strict XML tags (`<context>`), and streams the grounded prompt back across the stdio pipe to the agent to synthesize an answer.



3. LOW-LEVEL DESIGN (LLD)

3.1 Code System Directory Layout

The codebase enforces physical separation across all modules to prevent implicit dependency bleeding and ensure circular import safety.



3.2 System Interface Contracts & Component APIs

3.2.1 Core Configuration & Telemetry

- ``rag/config.py``: Manages path enforcements (`EMBEDDINGS_DB_DIR`) and hardcodes the structural embedding model target (`all-MiniLM-L6-v2`).

- ``rag/telemetry.py``: Mounts isolated file logger instances (`ingestion_logger`, `retrieval_logger`, `generation_logger`) to independent target files inside `rag/logs/` to eliminate cross-contamination during active debugging.

3.2.2 Ingestion Engine (`rag/ingestion/`)

- ``chunker.py`` (`load_and_chunk_json``): Formats raw JSON inputs into unified paragraph blocks enriched with domain prefixes (e.g., "MARPOL Regulatory Reference Code..."), outputting structured lists of text, unique string IDs, and metadata maps.
- ``vector_db.py`` (`PersistentVectorStore``): Mounts `chromadb.PersistentClient` and executes the database cache checking cycle to prevent redundant recalculation loops.

3.2.3 Retrieval & Generation (`rag/retrieval/` & `rag/generation/`)

- ``search_engine.py`` (`semantic_query_lookup``): Takes the user query string, converts it to a vector, calculates cosine similarity metrics against disk storage, and flags match items with a `HIGH` or `LOW` confidence rating.
- ``prompter.py`` (`build_grounded_compliance_context``): Aggregates retrieval elements, drops chunks that drift past the strict 0.55 math boundary limit, and builds the final text envelope using explicit XML grounding rules.

4. SYSTEM CONVERGENCE & EDGE-CASE PROTECTION

The system employs defensive code barriers across each processing layer to eliminate infinite agent loops, context bloat, and environmental failures:

- **LangGraph Loop Interruption:** A hard-counter evaluation check inside `should_continue_loop` forces a direct exit transition to `END` if the agent attempts more than 3 consecutive tool execution runs in a single turn.
- **Conversational History Isolation:** Thread tracking arrays are allocated dynamic identifiers (`maritime_evaluation_run_{index}`) for each query run, ensuring past context never leaks into subsequent evaluation loops.
- **Cross-Platform OS Stability:** Isolating the binary cache directory ensures that removing `rag/embeddings_db/` before packaging forces a fresh, error-free setup on the evaluator's operating system.

